SpotCast

Project report

SWMAD-01 Mobile Application Development

Author: Nuno Alves Gonçalves, study number 202600724 (the only member of the group)

Delivery date: 2026-10-29

Test user: see the Brightspace report

The test user exists in the Firebase project that the handed-in google-services.json points to.

1. App vision

People who spend time outdoors keep a mental list of places: a beach that gets waves in a westerly wind, a forest for a walk, a pier with a good sunset. A weather app gives them numbers for the nearest town. It does not say whether today is a good day to surf at that beach, or that the sunset photo is lost to clouds but the waves are good.

SpotCast turns the forecast for each saved spot into a plain answer: **Go**, **Maybe** or **Skip**, for each of five activities: a beach day, surfing, hiking, photos and a picnic. The app opens on a map of the user's spots, where every pin has the colour of today's answer. A bar on top switches the activity, or shows each spot's best activity for the day. Tapping a pin opens a card with the reason ("Clouds will hide the 16:51 sunset"), the answer for all five activities at that spot, a suggestion to switch when something else is better there ("Great for surfing"), and a strip with the next seven days. Spots are saved where the user stands, or anywhere by address or coordinates, and can be deleted after a confirmation.

2. Features and requirements

User stories

US1. Where to go today. As someone with a free afternoon, I want to see at once which of my spots are worth it today, so that I do not have to read seven forecasts. *Accepted when:* the app opens on a map with one pin per spot, green, amber or red for Go, Maybe or Skip, and a line that says how many spots are a Go.

US2. Plan by activity. As a photographer who also surfs, I want to choose what I want to do and be told when another activity suits a spot better. *Accepted when:* the bar offers Best today, Beach day, Surf, Hike, Photos and Picnic; the pins change colour with the choice; and a spot that is a Skip for the chosen activity but a Go for another one shows that activity with a Switch button.

US3. Why, and which day. As a user, I want to know why a spot is a Skip and which day this week is better. *Accepted when:* the card names the reason with the numbers behind it, and a strip of seven days shows the answer for each day and names the best one.

US4. Save a spot here or elsewhere. As a hiker, I want to save the place where I stand with one button, or a place I have not been to by its address or coordinates. *Accepted when:* the location permission is asked the first time; an address is looked up and shown as a pin under "Is this the right place?"; and only a confirmed position is used.

US5. Remove a spot safely. As a user, I want to delete a spot I no longer need, without deleting one by accident. *Accepted when:* holding a spot in the list or its pin on the map, the delete button on the card, and the details screen all open the same confirmation, and only "Delete" removes it.

US6. Spots follow the account; light or dark. A spot saved on one device shows on the other without a refresh, a user never sees another user's spots, and the theme can be System, Light or Dark, including the map.

How the mandatory requirements are met

- **Kotlin and coroutines.** All service functions are suspend functions. View models start them in viewModelScope; the forecasts of all spots are loaded in parallel, one coroutine per spot, so each pin gets its colour as soon as its own answer arrives.
- **Jetpack Compose.** All UI is composables with Material 3: Scaffold, navigation bar, filter chips, cards, lazy lists, dialogs, and an animated card over the map.

- **Compose Navigation.** One NavHost with seven routes. The details route takes the spot id as an argument. The back stack is cleared on sign in and sign out, and system Back closes the card on the map before it leaves the screen.
- **Four or more screens.** Today (the map), Spots, Add spot, Spot details and Profile, besides Sign in and Sign up.
- **Firebase Authentication.** E-mail and password, behind the interface AuthService.
- **Firestore.** Spots are documents in users/{uid}/spots, read through a snapshot listener so the map and list update live. Security rules let a user read and delete only their own spots, and accept a new spot only with the five fields the app writes and a name and note inside their size limits.
- **External HTTP API.** The Ktor client calls three endpoints without any SDK or key: the Open-Meteo forecast (temperature, rain, wind, cloud cover and sunset per day), the Open-Meteo marine API (wave height, for beach spots), and Nominatim to turn an address into a position. Each answer is mapped to a sealed class with a success case and named failures.
- **Additional technology.** GPS through the fused location provider, and a vector map drawn by MapLibre with the free OpenFreeMap tiles, recoloured at run time to the app's palette in light and dark. It needs no billing account and no API key.

3. UI and user flow

The app starts on Sign in, or directly on Today if Firebase still has a signed-in user. Today, Spots and Profile are the three tabs of the bottom bar. Add spot and Spot details open on top of a tab and have a back arrow. The colours follow one rule everywhere: green, amber and red always mean Go, Maybe and Skip, on pins, chips, the seven-day strip and the list.

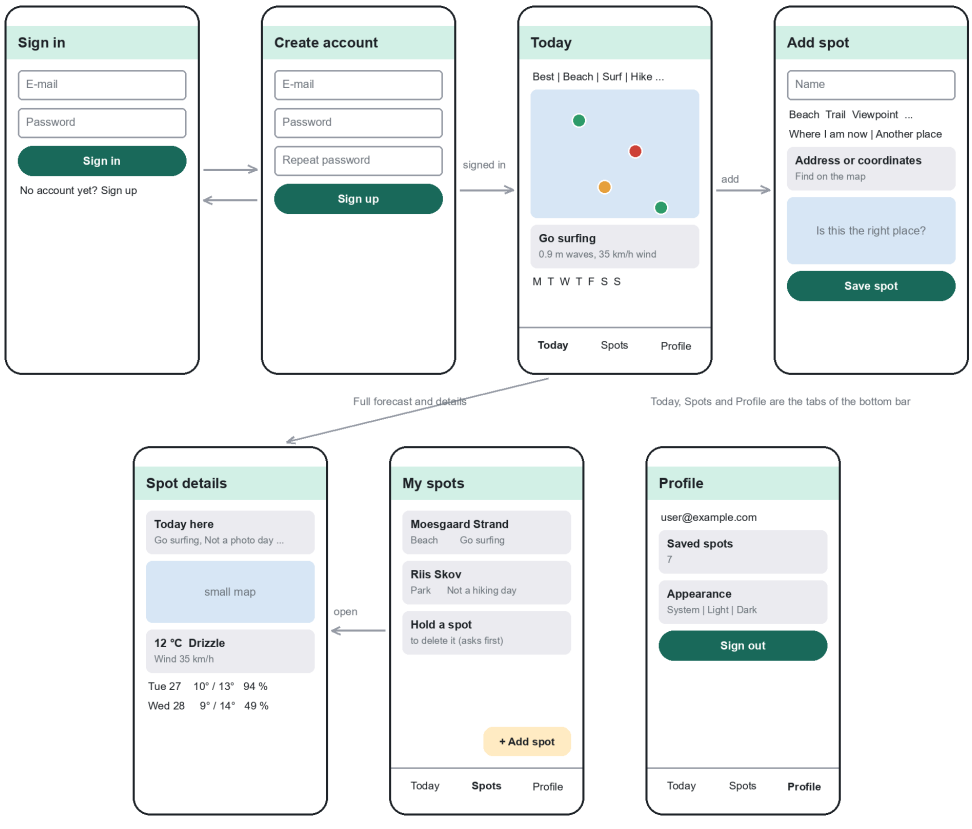


Figure 1. The screens and the navigation between them.

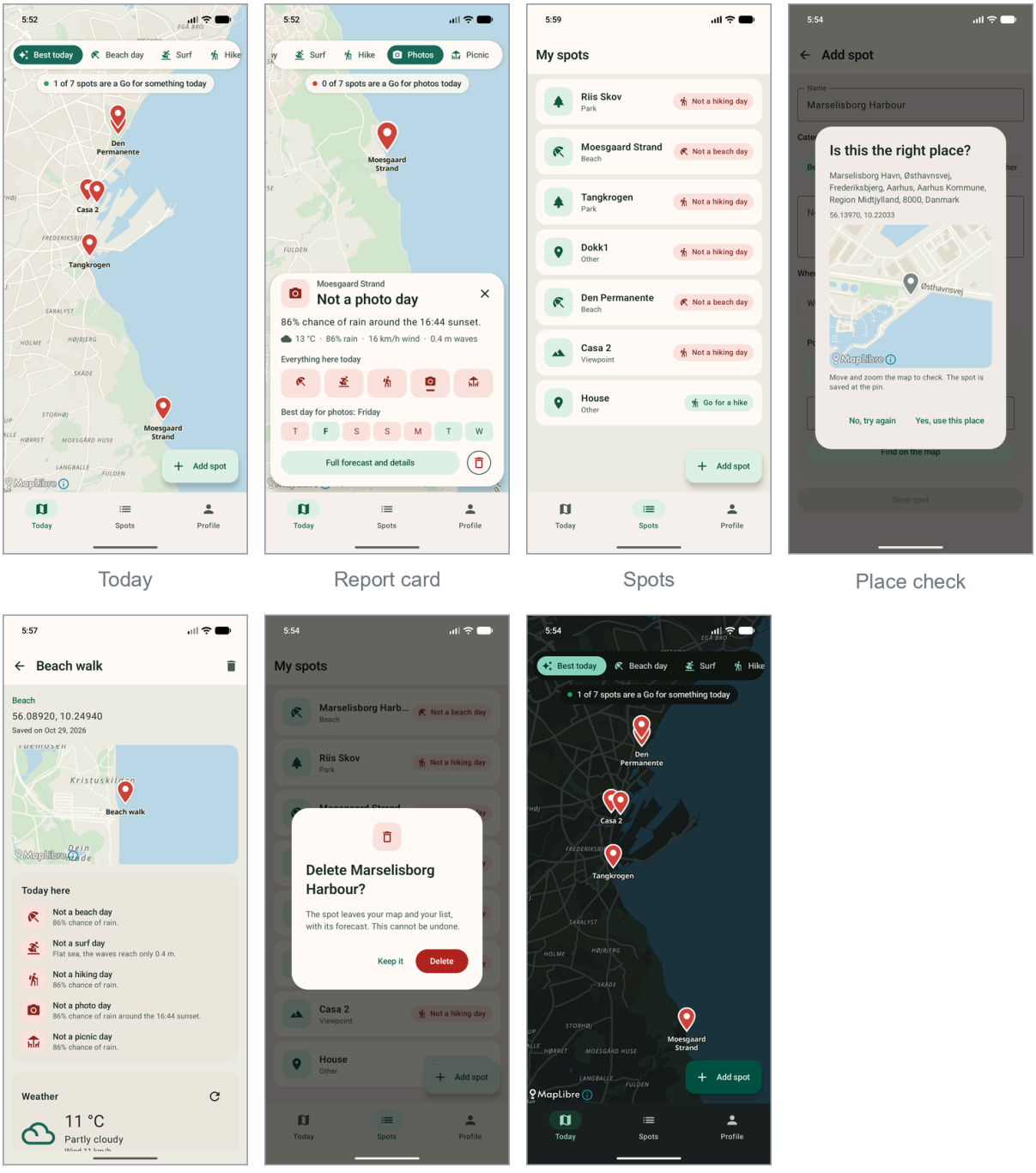


Figure 2. The app in the Pixel 10a emulator. First row: the Today map, the card for a spot in Photos mode (it names the best day of the week), the list, and the place check for a typed address. Second row: the details with the full report, the delete confirmation, and the Today map in the dark theme.

4. Components

The code is split in three packages. **ui** holds the composables and one view model per group of screens. **domain** holds plain Kotlin: the models and the rules that turn a day of weather into a verdict, with no reference to Android, Firebase or Ktor. **integration** holds one service per external component. Composables call view models, view models call service interfaces, and only the service implementations know about the external components. The map is the exception: MapLibre is a classic Android view, so it is wrapped in one composable, SpotMap, that loads its own tiles and draws the pins as a symbol layer.

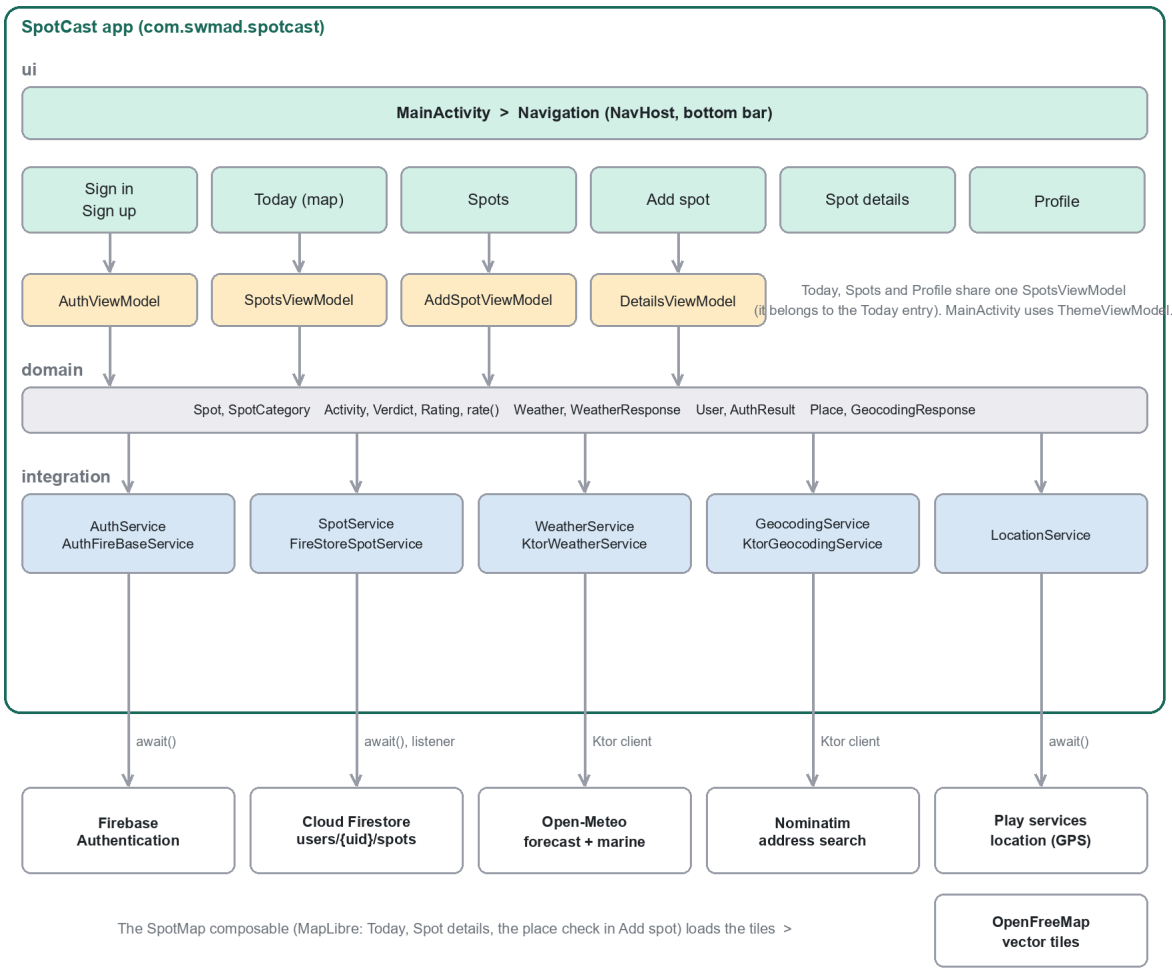


Figure 3. Components of the app and the external components they use.

The rules. The function rate(activity, day, nearWater) in domain/Activity.kt gives a Rating: the verdict and one sentence with the reason. The rules are fixed and the same for every user, for example surfing is a Go with waves of 0.8 m or more and wind under 40 km/h, and photos are a Go when broken clouds (25 to 80 percent) can light up the sunset. A Skip always names the first rule that failed. Thunderstorms are a Skip for everything. Only beach spots are rated for beach days and surfing, because the marine model snaps an inland point to the nearest sea. Being plain Kotlin, the rules are covered by JUnit tests.

Firestore data is read into SpotFS and mapped to the domain class Spot; the weather JSON is read into serializable DTOs and mapped to Weather. The theme is a setting of the device, held by ThemeViewModel and stored in SharedPreferences; the map reads the theme colours and recolours its layers when it changes. Today, Spots and Profile share one SpotsViewModel, which belongs to the Today entry of the back stack and is discarded at sign out.

5. Use cases

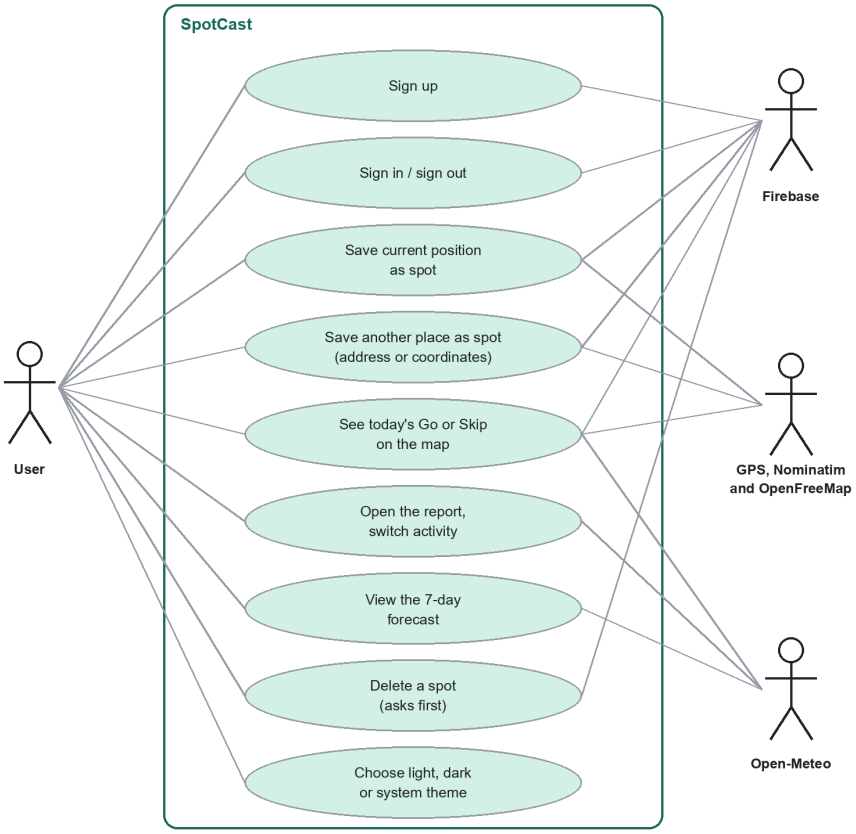


Figure 4. Use cases.

See today's verdicts. The user opens the app. SpotsViewModel listens to the spots in Firestore and, for each new spot, calls Open-Meteo (and the marine API for beach spots). Pins are grey while their forecast loads, then take the colour of the verdict for the chosen activity. The camera fits the largest group of spots. *Alternatives:* no spots, a card invites the user to add the first one; a forecast fails, that pin stays grey, the summary line says that there is no forecast, and a tap on the line or on the card of the spot asks again.

Open the report and switch activity. The user taps a pin. The card slides up, the map moves the pin above it, and the card shows the verdict, the reason, the conditions, the five activities, a suggestion if one is better, and the week. Switch, or a tap on another activity, changes the whole map. Back or a tap on the map closes the card.

Save a spot. In Add spot the user names the spot, picks a category and takes the GPS position, or chooses "Another place" and types an address or coordinates. A dialog with a map asks "Is this the right place?"; only yes fills in the position. *Alternatives:* permission denied, nothing found, no connection or invalid coordinates each give their own message.

Delete a spot. The user holds a spot in the list or its pin on the map, or presses the delete button on the card or in the details. A dialog names the spot and says the delete cannot be undone. "Keep it" closes the dialog; "Delete" removes the document, and the listener removes the pin and the card everywhere.

6. Conclusion

SpotCast ended up as an app that answers one question: where should I go today? My first version only saved spots and showed a forecast for each, which a weather app already does, so

I rebuilt it around a verdict: Go, Maybe or Skip for five activities, with the reason. All the mandatory parts are in: Compose with Navigation, coroutines, Firebase Authentication and Firestore, three calls with the Ktor client, GPS and a map.

Firebase took most of my learning time. I got stuck at the fourth step of the setup in the Firebase console until I had a written step-by-step list. google-services.json has to match the package name com.swmad.spotcast, and it stays out of the repository. Spots are stored in users/{uid}/spots, and a snapshot listener wrapped in a Flow updates the list and the map without a refresh. My first security rule gave each user everything under their own id. I later narrowed it to the spots path, with checks on the fields and their sizes, and tested it with the Firestore emulator on my computer.

In Android Studio I relied on Gradle, the emulator and Logcat. The version catalog (libs.versions.toml) keeps every library version in one file, so replacing the map library meant a few changed lines and some rewritten code. I tested nearly everything on the Pixel 10a emulator: I set its location for the GPS flow and switched on airplane mode to see how the app fails. Logcat showed the Ktor requests and my own Log.v messages. When I went offline it also showed the Firestore listener failing to resolve firestore.googleapis.com while the list stayed on screen. I did not use Compose previews. I checked each screen on the emulator.

Ktor and coroutines were new to me. Each service call is a suspend function, and each answer becomes a sealed class with a success case and named failures. A test with a broken server answer showed that one kind of exception was not caught and would have crashed the app. I added a catch-all that still lets a cancelled coroutine stop. The map took a detour. The course uses Google Maps, which needs a billing account, and I did not want to set one up. I used OpenStreetMap with osmdroid at first, then MapLibre with free vector tiles, which I could recolour for the light and dark theme. The change cost a rewrite of the map code and no money.

The main thing I learned is to test the failure cases. Everything worked online. The first offline run found a summary line that never stopped spinning and a map that stayed blank without a message. I would write those tests earlier next time.

7. Known bugs and issues

I ran the app on the Pixel 10a emulator against the real Firebase project: sign in, sign out, adding a spot by GPS, by address and by typed coordinates, the live list and map, delete from the list and from the details, the theme switch, and the app without a connection. These problems were found that way and are fixed:

- With no connection the summary line on the map kept spinning. It now says that there is no forecast, and a tap on it asks again.
- A failed delete would have replaced the whole list with an error. It now shows a message at the bottom and the list stays.
- An odd answer from Open-Meteo or Nominatim could crash the app. It is now an error message on the screen. Fake server answers cover this in the unit tests.
- The map could be rotated and tilted without a way back to north. Both gestures are off.
- The map stayed blank without a connection. After 8 seconds it says so and offers Try again.
- Each tab loaded all forecasts again. The three tabs now share one view model.
- The delete dialog closed when the phone was rotated.
- The first security rules allowed any field below the user's id. They now allow only the spots path, with checked fields and sizes.

These problems are still open:

- When the connection comes back, the forecasts do not reload by themselves. A tap on the summary line or on a spot loads them.
- If the map needs more than 8 seconds to start on a slow connection, the message shows for a moment and then goes away.
- The message for a failed delete was only tested in the view model. I could not make Firestore refuse a delete on the emulator, so I have not seen it on screen. The rotate and tilt fix is checked in the code and by a review, not with a real two-finger gesture.
- I did not test two devices at the same time. The security rules were tested with the local Firestore emulator, not against the live project.
- Pins that are very close overlap at low zoom, and a tap opens the top one. The map does not show the user's own position.
- The address search uses only the best match. A spot cannot be edited, only deleted, and a delete cannot be undone.
- The password typed in the sign-in form is kept in the screen state so it survives a rotation. A production app should not do that.
- Texts are in English only and are written in the code, not in string resources. The unit tests cover the rules, the services, the summary line and one view model, not the screens.